

P?S/8 PTUTIL

User Guide



P?S

Name: P?S/8 PTUTIL User Guide
Document: PQS8 PTUTIL User Guide.rtf
File[s]: PQS8 PTUTIL User Guide.PDF
P?S Index: O>R,O>R.70037OSTSHM0037TRDQ0037FTHCD
Security Level: 2
Location: NY
Date: 12-Jan-2018
Medium: PDF

1st Private Development Prerelease	September 2015
2nd Private Development Prerelease	October 2015
3rd Private Development Prerelease	November 2015
4th Private Development Prerelease	January 2018

P?S/8 PTUTIL

User Guide

© 1990, 2018 Copyright »CLA Systems

As of January 2018, the PTUTIL utility is still being tested in stages; certain routines still need to be added to make an entire functioning utility, as a bottom-up approach is being used due to the complexity of certain of the options. As has been used elsewhere, some functions started out as dummy routines merely to avoid assembly errors; temporarily alternate routines are being used to confirm the completed sections actually work as intended.

For example, instead of outputting to some combination of P?S/8 output files, the characters slated for output (past internal filters and conversions) are merely displayed on the console device. This allows confirmation that all lower-level routines are functioning properly. Certain program bugs to date were detected and fixed while debugging PTUTIL on the PEPS for windows SIMH simulator package.

Currently, there is no support for creating output from the contents of P?S/8 text files; in the scheme of things, this is a lower-priority function, and easier to debug (at least in theory). Certain internal design considerations (such as overall program size) will not be impacted by output design as much as all of the input parameters which can severely impact coding techniques. Note: At all costs, the program needs to be complete as loaded into memory in 4K. The current operating mode will be determined by carefully designed once-only initialization code processing the passed command-line switches. As such, certain memory areas must be carefully chosen to act as buffers instead of code pages after initialization. If overlays are required, the program will become unwieldy and incompatible with the R (or RU{n}) % (or R [or RU{n}] \$) program execution methods; this would cause the addition of PTUTIL to existing systems too difficult for most beginners.

This notice will be removed when a nominally complete program is being tested prior to release.

Contact:
CLASystems@gmail.com

CONTENTS

1.0	Introduction
1.1	Specific Hardware Requirements
1.2	Program limitations
1.3	ASCII text paper-tape format
2.0	P?S/8 keyboard monitor command formation when invoking PTUTIL
2.1	Overall command flow
2.2	Command-line option switches
2.3	Passed numeric value format
3.0	P?S/8 file formats supported by PTUTIL
3.1	TFS Directory file support in PTUTILL
3.2	Extended-length file support in PTUTIL
4.0	Using PTUTIL to punch paper-tapes from P?S/8 input files
4.1	PTUTIL command-line option switches used when punching paper-tapes
5.0	Creating P?S/8 output files from paper-tapes
5.1	PTUTIL command-line option switches used when reading paper-tapes
6.0	SIMH simulator and related issues
6.1	High-speed reader considerations
6.2	Host file sentinel character considerations
6.3	Parity bit issues
7.0	Additional PEPS for windows considerations
	Appendix A - Using PTUTIL without full installation
	Appendix B - Extended-length file support in P?S/8 Version 8Z
	Glossary of Terms

This page intentionally left blank.

P?S/8 PTUTIL User Guide

1.0 Introduction

Note: This document generally follows the conventions of the *P?S/8 Keyboard Monitor Command Guide* available separately. For new users unfamiliar with certain aspects of P?S/8, it may be useful to read this document or the *Glossary of Terms* below before proceeding.

P?S/8 now includes an ASCII paper-tape interchange utility named *PTUTIL*. As of P?S/8 Version 8Z, the current release is PTUTIL Version 1E.

Note: PTUTIL is being distributed as an add-on to existing copies of P?S/8; as such, the program may only be present on certain system release media. Appendix A contains information on using PTUTIL on P?S/8 Version 8Z systems should the program not be available within the System Programs Directory.

PTUTIL is used to process 8-level punched paper-tapes in standard ASCII text format on systems that include the requisite hardware.

Note: Binary paper-tapes punched in standard BIN or RIM format are handled separately by the P?S/8 *BIN* program documented separately.

Paper-tapes may be punched from the contents of existing P?S/8 SIXBIT text files or SIXBIT text files may be created from paper-tape input characters. The direction of transfer and choice of input (or output) paper-tape device is determined by command-line option switches passed to the program.

1.1 Specific Hardware Requirements

Beyond any normal system configuration to support P?S/8, PTUTIL requires the standard high-speed reader/punch hardware (device 01/02) or the low-speed reader/punch attached to the console terminal device (03/04); suitable models include the Teletype Model 33 Automatic Send and Receive (ASR) and the Teletype Model 35 ASR. The ASR device must conform to DEC specifications regarding reader run control.

1.2 Program limitations

If the source of the paper-tape input data is the low-speed reader, special handling of certain control characters (such as **Control-C**, **Control-S**, **Control-Q** and **Control-O**) will be disabled to prevent data garbling. As such, only devices that do not require reverse flow control can be supported. While this will always be the case on the supported Teletype models, PTUTIL may not be able to handle certain hybrid console devices that were never supported by DEC.

If any reader character echoing issues arise (such as the need for flow control), PTUTIL must be used in an operating mode that does not access the console output device. Command-line option switches are available to produce a variety of console output displays during reader input; If necessary, all console output can be disabled. However, if any of the options that echo input data on the console output device are in effect, it will not be possible to detect `Control-O` (which reverses the state of character echoing).

Additionally, immediate program abort by the detection of `Control-C` will not be possible until reader time-out has occurred.

None of these restrictions apply when paper-tape input is obtained from the high-speed reader; however, reader throughput can be degraded by echoing the characters on a low-speed console output device. The console output display rate must be of sufficient speed (typically exceeding 300 characters/second) to maintain reader throughput.

High-speed readers must maintain a minimum throughput speed to avoid slowing down to a small fraction of normal operating speed. This situation can easily become further degraded by poor maintenance practices; many high-speed readers become misadjusted with respect to the speed threshold adjustment. Since few applications echo reader input directly, many users are unaware of this problem. However, all related problems can be avoided by preventing character echoing (which allows the reader to run within its higher-speed performance range).

Note: If this situation becomes chronic, it is recommended diagnostic tests be performed on the high-speed reader station and/or interface.

1.3 ASCII text paper-tape format

Fully conforming ASCII text paper-tapes contain sections as follows:

a) Leader Section

The leader section consists of one or more frames with only the high-order bit punched. Many paper-tape editing programs tend to create long leader sections to assist the operator in positioning the paper-tape in the reader. (Note: PTUTIL ignores the leader section when creating P?S/8 output files. The first frame with any other code starts the data section.)

If PTUTIL is used to create ASCII text paper-tapes, the default leader (and trailer) length is 100 (octal 0144) frames. This value can be overridden with the `=xxx` command-line parameter to change the length to `xxx` frames as required. Setting the parameter to `=1023` will produce the longest (octal 1777) leader/trailer section permitted; attempts to use larger values will yield an error message.

Note: Using the `=0` parameter value will eliminate all leader/trailer frames. This may be useful when directing the output to the system console device.

b) Data Section

The data section generally consists of frames punched with (nearly) any ASCII character code value permitted. While the parity bit can be ignored, the specific frame value of only punching the high-order bit is not allowed in the data section to avoid anomaly with leader/trailer frames.

Most paper-tape editing programs create data section frames with the high-order bit set. This is used to create clear visual distinction from binary paper-tapes (which do not punch the high-order bit in the binary data section).

When PTUTIL is used to create ASCII text paper-tapes, all frames in the data section are subject to the parity option in effect. The disposition of the high-order bit will be determined by PTUTIL command-line option switches that affect frame parity. All four frame parity variants are available including mark parity (where the high-order bit is always set) which is used by default, space parity (where the high-order bit is always clear), odd parity (where the high-order bit is calculated to maintain an overall odd bit count) and even parity (where the high-order bit is calculated to maintain an overall even bit count). PTUTIL ignores the parity bit within all data sections of input paper-tapes.

Note: Using parity options other than the default mark parity will yield paper-tapes possibly unreadable due to violation of prevailing PDP-8 paper-tape standards. These options are provided for the express purpose of creating paper-tapes for export to other systems.

When creating P?S/8 output files, data frame parity is ignored. All frames in the data section are subject to conversion to P?S/8 SIXBIT text format as described in Appendix C of the *P?S/8 Keyboard Monitor Command Guide* available as a separate document.

Note: At the time of this writing, the above-referenced document is being updated to describe changes to recent releases of the prevailing inter-system conversion programs including PTUTIL. The only issues under review concern the disposition of the seldom used `@` and `■` characters; in previous releases of *OS8CON* and related conversion programs, these characters were ignored. In future releases of these conversion utilities, the characters will be converted to `#` and `■` respectively.

c) Trailer Section

The trailer section consists of one or more frames with only the high-order bit punched. Certain paper-tape system programs create lengthy trailer sections to assist the operator in positioning the paper-tape in the reader.

Note: In some instances, there are multiple data sections within a single paper-tape; the trailer of one section also serves as the leader of the next section, etc. PTUTIL supports command-line options to enable the processing of multiple input sections or multiple tapes if required.

If PTUTIL is used to punch ASCII text paper-tapes, the default trailer length is 100 (octal 0144) frames. This value can be overridden with the `=xxxx` command-line parameter to change the trailer and (leader) length as required.

Note: When creating P?S/8 output files, PTUTIL closes the current output file on the first occurrence of a trailer character; all trailer characters are otherwise ignored. The current output file will also be closed if there is no further paper-tape input as determined by paper-tape reader device time-out. As such, the trailer section is considered optional. If multiple-tape options are in effect, the reader can be restarted as necessary; the operator must reposition (or change) the paper-tape in the reader device to an appropriate leader section to allow further input during the overall conversion session.

d) Sentinel Character

A final frame may be punched with the `Control-Z` character code to indicate overall end of frame data. (Note: within the prevailing operating systems supporting this convention, the sentinel frame character code is 032 octal; the high-order frame bit is cleared as the sentinel character is not part of the trailer section.)

Not all operating systems support the sentinel character frame. PTUTIL can create paper-tapes with the sentinel character suppressed by using the `'/Z'` command-line option switch. It is recommended that the `Control-Z` character be used in conjunction with a shortened trailer section (or perhaps no trailer section). Systems that support this convention generally expect the sentinel character (or perhaps will modify the resultant file to conform to local system standards).

The method described above applies to (at least) the TOPS-10 and OS/8 operating systems. Other systems may use the convention of `Control-Z` termination without internal leader or trailer characters (such as the P?S/8 SHELL currently under development).

Judicious use of the `=xxxx` command and the `'/Z'` command-line option switch should allow accommodation of virtually any operating system requirement with regard to file termination of punched ASCII text paper-tapes.

2.0 P?S/8 keyboard monitor command formation when invoking PTUTIL

Note: P?S/8 supports several command prompt variations. The prompt examples used throughout this document are analogous to those used in TOPS-10 and OS/8. This will help facilitate comparisons between the various operating systems. `<SP>`, `,` and `<HT>` characters are freely allowed between command elements (including files within file lists) to enhance command readability and are otherwise ignored.

Command-line formation in P?S/8 allows options to be passed to system programs in the same overall manner as OS/8 CCL commands (with a few minor variations and several unique extensions).

2.1 Overall command flow

The P?S/8 keyboard monitor allows command formation in either file specification order as shown below; this allows the user to work in an intuitive environment regardless of prior experience. Commands are generally similar to various concise command implementations found in other DEC operating systems.

Typical command examples to invoke PTUTIL (in slightly simplified form) are structured as follows:

- a) Create P?S/8 output files from paper-tape input.

`.(optional R or RU{n}) PTUTIL output files < option switches`
or

`.(optional R or RU{n}) PTUTIL > output files option switches`

- b) Create paper-tapes from P?S/8 input files.

`.(optional R or RU{n}) PTUTIL < input files option switches`
or

`.(optional R or RU{n}) PTUTIL input files > option switches`

Note: Attempting to pass a mixed set of input and output files to PTUTIL will cause an error message and program exit.

within keyboard monitor execution-class commands specifying only input files, the use of < and > characters is optional; these are often used to clarify command variations by delineating command sections while maintaining command symmetry. The use of these characters is recommended within P?S/8 *BATCH* files to enhance readability of deferred commands.

2.2 Command-line option switches

The P?S/8 command-line supports option switches passed in a manner consistent with many DEC operating systems. Two variant forms are supported as follows:

a) Individual option switches each preceded by / such as:

/F /U /V

b) Grouped switches preceded by (such as:

(FUV)

The trailing) is optional unless needed to delimit the option group from another command section.

2.3 Passed numeric value format

A single numeric value can be passed to any system program using the =xxxx syntax where xxxx is any value of up to four digits in the range of 0 through 4094 (0000 through 7776 octal). The explicit value 4095 (7777 octal) is not allowed; however, that value is passed to the program to indicate the explicit form of =xxxx was not used within the command.

If the =xxxx parameter is not explicitly passed to PTUTIL, the number of leader/trailer frames punched when creating paper-tape output is set to the default value of 100 (0144 octal). Passed values larger than 1023 (1777 octal) are not allowed and will yield an error message.

Command-line option switches (and the =xxxx parameter if required) can be placed anywhere in the command line after the program name (in this instance *PTUTIL*) and may be stated redundantly.

For more information about P?S/8 command-line variations, consult the *P?S/8 Keyboard Monitor Command Guide* available as a separate document.

3.0 P?S/8 file formats supported by PTUTIL

The PTUTIL paper-tape interchange utility supports two different P?S/8 SIXBIT text file formats:

- a) Fixed-length files that are compatible with the Tiny File System (TFS) directory structure,
- b) Extended-length files. Note: Extended-length files are only available as individual files on each logical device unit. Appendix B contains additional information regarding the use of extended-length files.

3.1 TFS Directory file support in PTUTIL

TFS Directory files are usually created by the P?S/8 keyboard monitor using built-in text editing commands; each file is 2048 12-bit words. In general, each file will tend to contain up to 3/4 of this capacity as six-bit bytes in the P?S/8 SIXBIT text format as described in Appendix C of the P?S/8 Keyboard Monitor Command Guide available as a separate document. This corresponds to about 3000 characters per TFS Directory file depending on average line length and the internal line pointer and line number data.

Note: Each line in the file has an associated pointer to the start of the line in memory and a permanent line number. This allows rapid file editing by the P?S/8 keyboard monitor, which includes a 2048 word edit buffer in memory to hold the entire file while performing all editing operations; file updates are not made permanent until the file is written back to the storage device.

When PTUTIL is used to create TFS Directory output files, each file includes automatically-generated line numbers starting at 100 and incrementing by 10 for each additional line. Files are closed when a `<FF>` character from the paper-tape is being processed and there is less than a defined amount of free space (for potential future editing) left in the file. Should there actually be insufficient space to store the latest line obtained from the paper-tape input, the current file will be closed and the next output file will be opened. The process continues until the entire paper-tape is processed (or there is an error caused by insufficient output files passed to PTUTIL).

PTUTIL can be passed up to 17 TFS Directory output files which corresponds to about 50,000 text characters of paper-tape input from a single session.

Note: While this might be considered a large quantity of input for practical use of ASCII text paper-tapes, P?S/8 easily handles larger text projects if necessary. This would generally be accomplished by additional processing as documented within the P?S/8 Keyboard Monitor Command Guide and is beyond the scope of this document.

An error message will be issued if insufficient output files are supplied in the command invoking PTUTIL. As such, it is generally recommended to supply more output files than contemplated. PTUTIL displays a summary of how many output files were actually written; any unused files can be repurposed.

3.2 Extended-length file support in PTUTIL

PTUTIL optionally supports P?S/8 extended-length SIXBIT text files. This file format is identical to the TFS Directory SIXBIT text file format in terms of internal text characters; however, no line numbers (or related information) exist within these files. As such, individual extended-length output files can typically be created up to the storage limit on each logical device unit selected for file creation.

Appendix B describes the past, current and future implementation of extended-length files as it applies to P?S/8 Version 8P and newer releases.

Note: A surviving release of P?S/8 Version 8T for TC08 DEctape is available within the standard release of the P?S Extended PDP-8 Simulator (PEPS) for windows package; this obsolete system is primarily used to show the progression of P?S/8 development over the years.

As a practical matter, a maximum of eight extended-length files may be available for use with extended-length SIXBIT text files on specific configurations; fewer logical units supporting extended-length files will be available on more modest systems depending on hardware considerations.

When PTUTIL is writing to extended-length output files, the current file will be closed if the latest line of paper-tape input cannot be processed without exceeding file limits. An error message will be issued if insufficient output files were passed. PTUTIL displays a summary of how many extended-length output files were actually created during paper-tape processing; unused files can be repurposed.

When PTUTIL is used to create ASCII text paper-tapes, all line number information (if any) is ignored. As such, TFS Directory and extended-length input files can be freely mixed. PTUTIL can be passed up to 17 input files. On sufficiently large P?S/8 configurations, this corresponds to as much as 8,000,000 paper-tape output frames (not counting leader and trailer). (Note: For all paper-tape frame tallies and summaries, PTUTIL calculates the various paper-tape frame counts in 24-bit unsigned integer arithmetic.)

while these numbers may at first appear to be preposterously large, one of the design goals of PTUTIL is to interface P?S/8 to external host files when used in a simulated environment such as the PEPS PDP-8 simulator package for windows; all inter-system file interchange is via paper-tape simulation. For the largest PDP-8 programs written to date, this should be adequate for any given project.

Note: Huge programming projects tax all PDP-8 development systems whether running on real PDP-8 hardware or implemented via simulation.

P?S/8 can provide superior support for large projects throughout the entire development phase. In general, large-scale programs are intended as stand-alone bootable applications.

Note: By and large, common PDP-8 development operating systems are unsuitable for the purpose of resource management within large application-specific systems.

4.0 Using PTUTIL to punch paper-tapes from P?S/8 input files

When P?S/8 input files are passed to PTUTIL, paper-tape frames are punched on the selected output device. Up to 17 (0021 octal) P?S/8 input files can be passed to PTUTIL; the output will be a continuous stream of paper-tape frames taken from the concatenated contents of all input files.

Note: In this operating mode, TFS Directory and extended-length input files may be freely mixed.

Console message prompts will generally be issued at the start and finish of the paper-tape punching session to guide the user through the process. If the output device is the low-speed punch, no finishing message can be issued; however, the program will wait for a character to be pressed on the console keyboard when the punch should be enabled (and later disabled). This is necessary to prevent message text from being inadvertently punched to the output tape.

4.1 PTUTIL command-line option switches used when punching paper-tapes

Command-line option switches that apply when punching paper-tape output are as follows:

/A (*Automatic mode*) All prompts for user input are eliminated. The selected paper-tape punch hardware must be ready before invoking PTUTIL.

Note: Automatic mode can be useful within P?S/8 BATCH sessions designed to carry out repetitive tasks.

`/F` The default paper-tape output device is the low-speed punch of the Teletype model 33 ASR or model 35 ASR console device. If the `/F` command-line option switch is set, the output device is changed to the DEC standard high-speed punch (device 02).

Note: High-speed punch support is subject to hardware time-out.

If the high-speed punch is not available, the output will be redirected to the console device. Unless the `/S` command-line option switch is set (for silent-running mode), unexpected console message text might be punched within the paper-tape output.

`/S` (*Silent-running mode*) If the `/S` command-line option switch is set, all console output is suppressed with the exception of serious error messages. Verbose mode (setting the `/V` command-line option switch) will be ignored.

Silent-running mode can be useful within P?S/8 BATCH operations that might otherwise create unwanted console output. Since P?S/8 BATCH can suppress its own console output messages, a carefully crafted BATCH file can be designed to punch a series of paper-tapes on the low-speed punch with all unwanted messages eliminated.

`/U` (*Upper-case mode*) If the `/U` option switch is set, all console messages will be printed in upper-case characters. (The default is mixed upper and lower case.)

Note: This option is only required if the console terminal does not support lower-case input.

`/V` (*verbose mode*) When the `/V` command-line option switch is set, certain console messages will be embellished to provide additional program management guidance and/or program statistics normally suppressed.

Note: If silent-running mode is in effect (by setting the `/S` command-line option switch), verbose mode will be ignored.

`/Z` Setting the `/Z` command-line option switch inhibits the sentinel `Control-Z` character. This can be useful when punching multiple section paper-tapes. PTUTIL would be invoked with the `/Z` command-line option switch set except for the final execution step that creates the sentinel character.

Note: When creating output oriented towards certain operating systems (especially when running under the SIMH PDP-8 simulator), host operating system conventions might warrant preventing the `Control-Z` character in the punched paper-tape output stream.

/0 Setting the /0 command-line option switch enables space parity generation in the data section(s) of paper-tape output (overriding the default mark parity); this can be useful when

creating output for export to systems that require space parity format.

Note: PTUTIL does not check parity when reading paper-tape data.

/1 Setting the /1 command-line option switch enables odd parity generation in the data section(s) of paper-tape output (overriding the default mark parity); this can be useful when creating output for export to systems that require odd parity format.

Note: PTUTIL does not check parity when reading paper-tape data.

/2 Setting the /2 command-line option switch enables even parity generation in data section(s) of paper-tape output (overriding the default mark parity). This can be useful when creating output for export to systems that require even parity format.

Note: PTUTIL does not check parity when reading paper-tape data.

When using the =xxxx parameter and punching paper-tape data, the default length of the leader and trailer sections is 100 (0144 octal) frames with only the high-order bit set. The =xxxx parameter can be set to any value in the range of 0 through 1023 (0000 octal through 1777 octal) to override the default length.

Note: Setting =0 will eliminate the leader and trailer sections.

5.0 Creating P?S/8 output files from paper-tapes

In this operating mode, paper-tape frames will be converted into the contents of P?S/8 output files. There are two variants to this operating mode:

- a) Output is created as the contents of up to 17 TFS Directory output files. Sufficient files must be passed to avoid data overflow. The count of output files created will be available in the final program summary. Unused files are ignored and can be repurposed. This output mode is used when the '/E' command-line option switch is not set.

- b) Output is created as the contents of up to eight extended-length output files. Sufficient files must be passed to avoid data overflow.

Note: Smaller P?S/8 configurations may only support fewer than eight logical device units limiting the maximum output size. Additionally, the amount of space available to an extended-length file is subject to several configuration issues including the underlying size of the storage device. This output mode is used when the '/E' command-line option switch is set.

5.1 PTUTIL command-line option switches used when reading paper-tapes

Specific command-line options that apply are as follows:

- /A (*Automatic mode*) All prompts for user input are eliminated. The selected paper-tape reader hardware must be ready before invoking PTUTIL.

Automatic mode can be useful within P?S/8 BATCH sessions designed to carry out repetitive tasks.

- /B (*Buffered mode*) By default, all character frames received on the high-speed reader are available to PTUTIL. If the /B command-line option switch is enabled, a double-buffered approach is enabled. This is used in certain PDP-8 systems on the theory that the apparent final character received from the reader is a junk frame which must be suppressed. See Section 6.1 for more information regarding using PTUTIL when running under a PDP-8 simulator.

- /D (*Display mode*) If the /D command-line switch is set, all characters from the paper-tape data section(s) are displayed as read (processed according to P?S/8 character conversion rules).

Display mode should be used sparingly as throughput may be dramatically reduced. Echoing the data section characters can be helpful to ensure the intended paper-tape is being read. Pressing the **Control-O** key will reverse the current state of character echoing.

Note: Due to hardware restrictions, **Control-O** detection is not available when the low-speed reader is used for paper-tape input. Program abort can only be accomplished by manually disabling the low-speed reader to cause a hardware time-out. The operator can press the **Control-C** character to exit the program.

Future releases of PTUTIL may be capable of determining if the Logical Console Overlay is in effect and addressing an alternate hardware device for console input; this would allow restoration of these functions controlled from the alternate hardware input device.

/E (Extended-length file mode) Setting the */E* command-line switch indicates the use of extended-length output files. The default operating mode uses TFS Directory output files.

/F The default paper-tape reader is the low-speed reader of the Teletype model 33 ASR or model 35 ASR console device. If the */F* command-line option switch is set, the input device is changed to the DEC standard high-speed reader (device 01). See the description of the */B* command-line option switch above for related information.

/M (Multiple tape mode) If the */M* command-line option switch is set, multiple input tapes may be processed. The initial user guidance is identical to single-tape mode (and subject to the same modifications such as silent running mode). When a trailer frame is encountered, the user is prompted for an additional input tape. Pressing *Control-Z* ends the overall reader session; position the next tape section in the reader then press any other key (which will not be echoed) to continue reading additional tapes.

Note: The latest output file is not automatically closed unless *Control-Z* is pressed; the contents of input paper-tapes are logically concatenated together as if one continuous frame sequence. An indefinite number of paper-tape sections can be read as long as the converted input can be stored to the sequence of P?S/8 output files supplied on the command line.

If silent running mode is in effect, guidance messages are suppressed; however, PTUTIL still requires appropriate keyboard input to proceed.

/N If display mode is in effect, character echoing will be enhanced to include page headers; the output format is similar to output of the P?S/8 *PRINT* program. This can serve as a log record of the paper-tape conversion when transitioning away from the paper-tape operating system to P?S/8. Page separation will include tear-off lines suitable for use on a Teletype Model 33 terminal using roll paper.

/P If display mode is in effect, character echoing will be enhanced to include page headers; the output format is similar to output of the P?S/8 PRINT program. This can serve as a log record of the paper-tape conversion when transitioning away from the paper-tape operating system to P?S/8. Page separation will use <FF> characters only supported on console terminals such as the Teletype Model 35 or the LA36 DECWriter II.

/Q If the /Q command-line option switch is set, the conversion process will check for additional space in the file when a <FF> character is encountered in the paper-tape input stream. It may be desirable to include entire edit pages within the same file. However, this can lead to the need for additional output files.

/S (*Silent-running mode*) If the /S command-line option switch is set, all console output is suppressed with the exception of serious error messages. Verbose mode (setting the /V command-line option switch) will be ignored.

Note: Silent-running mode can be useful within P?S/8 BATCH operations that might otherwise create unwanted console output. Since P?S/8 BATCH can suppress its own console output messages, a carefully crafted BATCH file can be designed to convert a series of paper-tapes with unwanted messages eliminated.

/U (*Upper-case mode*) If the /U option switch is set all console messages will be printed in upper-case characters. (The default is mixed upper and lower case.)

Note: This option is only required if the console terminal does not support lower-case input.

/V (*verbose mode*) when the /V command-line option switch is set, certain console messages will be embellished to provide additional program management guidance and/or program statistics normally suppressed.

Note: If silent-running mode is in effect (by setting the /S command-line option switch), verbose mode will be ignored.

/X when the /X command-line option switch is set, special handling of the <FF> character is disabled; the <FF> character will be converted to a P?S/8 end-of-line sequence.

Note: Setting the /X command-line option switch causes the /Q command-line option switch to be ignored.

6.0 SIMH simulator and related issues

When P?S/8 is run under a PDP-8 simulator, host system file conventions may influence the preferred command-line options used when running PTUTIL.

6.1 High-speed reader considerations

MS-DOS and windows-based simulators use file systems that maintain a byte count for each file. As such, there is no sentinel **Control-Z** character requirement; files maintained in the host system cannot be expected to include the sentinel character. When these files are passed to P?S/8, the end of input will of necessity be obtained by high-speed reader time-out.

Certain PDP-8 programs (including the standard high-speed reader device handler within OS/8) use double-buffering of the input frames expecting the final frame detected to be junk data; the last character frame will not be returned to the calling program. As such, in OS/8, the last character of the simulated high-speed reader input is never made available.

The default operation of PTUTIL is to allow every character detected to be available as an input character frame. Enabling double-buffering with the '/B' command-line option switch will prevent PTUTIL from processing the last character frame and is generally not recommended.

6.2 Host file sentinel character considerations

When punching paper-tape output, a designated host file stores all frames sent; if PTUTIL is running under the windows file system, all files are maintained with exact byte counts; as such, PTUTIL may be run with the '/Z' command-line switch set to suppress the **Control-Z** sentinel character. (Note: Depending on program specifics, the **Control-Z** sentinel character may appear as an unwanted artifact of the punching operation when performing host editing operations.)

When running PTUTIL under other host operating systems, the exact output file requirement can be adjusted for local host file system conventions as required.

6.3 Parity bit issues

It may be necessary to use the '/O' command-line option switch to enable space parity in all output data frames. Many systems support extensions to the seven-bit ASCII character set; as such, setting the high-order bit may cause misinterpretation as an extended character instead of the intended seven-bit ASCII value. This could include leader/trailer characters which can be suppressed as necessary using the =0 parameter setting.

Note: Information on the various SIMH simulator implementations is available elsewhere as part of various SIMH document collections.

7.0 Additional PEPS for windows considerations

As discussed in Section 6 above, it is possible there can be artifacts remaining in files transmitted to the windows environment as punched paper-tape character streams.

In the PEPS for windows package, commands are available to open and/or edit these files; artifact problems present are generally immediately obvious. To eliminate any and all such problems, the PEPS package includes a file-stripping utility that removes all known artifacts from either the high-speed punch output or the device 66 lineprinter output.

As such, the file-stripping utility can be used to eliminate all problems within ASCII text files.

Note: The stripping utility can also eliminate problems with files obtained externally from such as unix systems and/or Internet sources. Once stripped, text files conform to PDP-8 conventions and can be sent to the simulated PDP-8 environment with all attendant problems removed.

Additional information on the PEPS for windows package version of the SIMH PDP-8 simulator is available from the author of both this document and the PEPS package: Charles Lasner.

Appendix A - Using PTUTIL without full installation

PTUTIL is a recent addition to later releases of P/S/8 Version 8Z; it will be included in all future releases as a standard system program.

Users familiar with the P?S/8 System Programs Directory can add PTUTIL to existing systems where PTUTIL is not installed (assuming sufficient free blocks are available). PTUTIL can also be run as an impromptu system program as described below; the binary files created by assembling the PTUTIL source files must be available on an accessible logical device unit.

Note: Logical unit 1 is used in this example; appropriate changes can be applied as needed.

Console output

Comments

.	Keyboard monitor prompt.
.CA:1	CA{talog} command for unit 1.
BPTUT1 BPTUT2 README RDMETU	Contents of TFS Directory on unit containing binary files.
.LC:0	Back to system unit to avoid dependencies on data unit.
.GET BPTUT1:1 BPTUT2	Create program image.
	System restarts here.
.R \$ (Files and other parameters)	Particulars same as used when PTUTIL is normally installed.

The R (or RU{n}) command specifying the program name \$ (or %) executes the default core image as setup by the latest GET execution or equivalent. (Note: This will function permanently unless some other system usage or command changes the contents of the % and \$ files. As necessary, the GET operation can be repeated to refresh the default program image.)

PTUTIL has been designed to be (nearly) compatible with the technique described above to allow quick addition to existing systems; this execution mechanism alternative was introduced in P?S/8 Version 8Y and is supported on all newer releases. The majority of conforming system programs can be supported using this tentative mechanism in lieu of inclusion within the System Programs Directory (unless overlay structures are required).

Appendix A (continued)

Usage restrictions.

When using PTUTIL to create ASCII text paper-tape output, the default leader/trailer length (surrounding the content obtained from all input files) is 100 (0144 octal) frames. This value can be changed using the =xxxx parameter which is normally expressed in the current keyboard monitor radix (which is generally decimal). However, the execution attribute to maintain the current radix is only available to programs located in the System Programs Directory. (The default value is to force octal radix). As such, any usage of the =xxxx parameter as described here must be expressed in octal.

End of Appendix A

Appendix B - Extended-length file support in P?S/8 Version 8Z

Some form of extended-length file support has (nearly) always existed within P?S/8. The internal file format (described elsewhere) is ASCII SIXBIT text. These files are created by PTUTIL and other inter-system conversion utilities (such as *OS8CON* and *L6DCON*); however, the associated directory structure has gone through several partially functional (and incompatible) implementations over time. The current structure will be scrapped and logically replaced with the P?S/8 SHELL (which is expected to be an integral component of P?S/8 Version 9B).

Note: P?S/8 Version 9A (the next designated version) will largely be a cleanup release to remedy a number of issues that have been discovered over time. Small changes will be made to many programs to achieve internal improvement. From a usage standpoint, little will be changed with the exception of the specifics of extended-length files access.

At present, P?S/8 Version 8Z includes the vestigial Extended-length File Directory structure that only provides the rudiments of extended-length file support with serious issues as follows:

- a) Extended-length filenames are currently not supported. Instead, an existing TFS Directory file must be specified which is the highest (by storage block number) within the TFS Directory.

Admittedly, this implementation is seriously flawed. Users must be aware of and comply with this restriction; improper usage (without any proper safeguards) can easily destroy files further up the device (residing on higher storage blocks). The proper filename to provide (in P?S/8 Version 8Z) is the last filename displayed by the CA{TALOG} command output on the system console.

- b) The Extended-length Files Directory structure currently only serves a single purpose, which is to establish the high limit block on each logical unit. This value is used to ensure extended-length file access is limited to intended blocks.
- c) The only way to rename extended-length files is manually patching the filename characters (using P?S/8 *DUMP* or *BLKODT*).

Note: P?S/8 Version 8T (and similar releases) supports the first filename in the Extended-length Files Directory. However, the limit block is determined by the console switch register while running *BIGCON* (a forerunner of *OS8CON*). The current implementation in P?S/8 Version 8Z is a stop-gap improvement until a proper remedy is released.

Appendix B (continued)

- d) The rest of the Extended-length Files Directory structure is currently unused. It was originally created for a theoretical attempt at a multiple-file directory structure that was never fully implemented. This has since been scrapped in favor of the P?S/8 SHELL which is currently under development.

As such, utilities such as PTUTIL check to see if the '/E' command-line option switch is set; however, no validation can be performed to determine if output file handles reference extended-length file blocks. As described above, this is currently impossible; the passed block value merely shows the presumed highest block(s) actively used in the TFS Directory. Files will be created from the passed block value through as much as all the rest of the blocks on the storage device as indicated in the Extended-length Files Directory. Thus, the user is warned to always pass the highest actual filename in the TFS Directory to avoid file destruction.

All analogous conversion programs that create P?S/8 extended-length output files reserve internal initialization code space. When feasible, the space will be used for future implementation where extended-length output files are confirmed to start outside of the scope of the TFS Directory; appropriate validation can be applied (including appropriate error messages). When this is operational, the temporary kludge of supplying the starting block of the highest-block existing TFS Directory file will cease to function; safe and proper support of the individual extended-length file will replace the current risky implementation.

The Extended-length Files Directory block structure will be removed from P?S/8 (ultimately to be repurposed.) This will be replaced by a mechanism that directly defines the block limits of the individual extended-length file within the System Programs Directory.

In future P?S/8 releases, it will be necessary to use the R (or RU{N}) form of program execution to access extended-length files. This is in keeping with the original intention of P?S/8 extended-length file support; all proposed support mechanisms require sufficient code space in memory that cannot coexist with the keyboard monitor edit buffer.

Note: The keyboard monitor provides support routines for the execution-class code sections that are brought into memory, including the concise execution mechanism that overlays a portion of the keyboard monitor. This overall scheme is especially useful to certain utility programs that interact directly with the keyboard monitor in various ways. Certain programs are designed to post-process the command line for additional information; a good example is the P?S/8 *SET* utility. The entry for the *SET* program in the System Programs Directory indicates limited parsing by the execution-class command processor to allow the *SET* utility to process commands via its own syntax parsing routines.

Appendix B (continued)

Historically, in all relevant implementations of P?S/8, a distinction has always been made between supporting the first physical half of the System Programs Directory (the first two blocks) and supporting the entire directory due to specific resource issues.

The concise form of program execution (where merely the program name is invoked causing it to be executed) restricts the entry for the target program to be located within the first half of the System Programs Directory. This allows the P?S8 keyboard monitor to overlay the code that supports the execution and the first half of the System Programs Directory over a portion of the keyboard monitor (containing routines that are not used during the command-line processing).

Note: Where relevant, the displaced portion of the keyboard monitor will be restored by the target program after the command is processed. This also occurs if there was a program lookup failure or other command parsing problem. This is important in the case of programs such as the *FIND* and *CHANGE* utilities which interact with the keyboard monitor and the edit buffer. Most program execution commands merely lead to loading a program image into some portion of user memory (locations 00000-07577) which ultimately exits to location 07600 to reload the keyboard monitor; generally, no further interaction with the keyboard monitor is required unless the program is performing an abort operation.

The advantage of this form of execution is the preservation of the edit buffer which is crucial to the execution of certain overlaying utilities (including P?S/8 *BATCH*) which interact with the edit buffer contents. When PTUTIL is invoked without extended-length output files, this is a suitable means for initiating the utility.

One reason for using the R (or RU{N}) form of execution is to allow system program names that are anomalous to keyboard monitor commands. Additionally, since the second half of the system programs directory is fully accessible, this allows an upper limit of over 80 system programs; the concise execution form limit is 42 system programs (as stored in the first half of the System Programs Directory).

All program entries contained in the second half of the Systems Program Directory must, of necessity, not depend on the preservation of the edit buffer.

Note: This is generally the case, considering how few programs actually require edit buffer preservation. For most system programs, the lower overhead of the concise form of execution is a welcome improvement in performance; however, this is not an actual requirement.

Appendix B (continued)

As a practical matter, no existent P?S/8 release has ever fully occupied the first half of the System Programs Directory; the typical overall system program count is 30 (or less) unique entries. (Certain entries are merely aliases to other programs in the System Programs Directory.) This has partially been a result of ensuring most system program names are not in conflict with the keyboard monitor commands. For the average user, there are few (if any) additional programs that warrant placement in the System Programs Directory.

Generally, most programs are executed as TFS binary files loaded by the *BIN* utility. As such, there haven't been enough system programs to justify the full length of the System Programs Directory.

Note: Variant system releases can make these system programs entries meaningful; for example, numerous diagnostic program images could be added to a customized P?S/8 release.

Thus, it has always been intended to require these restrictions to access extended-length output files (as well as the entire System Programs Directory). The edit buffer memory locations are overlaid; a complete keyboard monitor reload is required if various errors are detected while parsing the command. Until the solution described below is implemented, the workaround for P?S/8 Version 8Z (as described above) must be used to access extended-length files.

Extended-length files will be formally redefined in future releases of P?S/8 starting with Version 9A. Each logical unit will support one extended-length file; supported file formats will not change. As a practical matter, the extended-length file starting block will be exactly 16 (0020 octal) blocks past the best-case workaround of supplying the highest possible TFS Directory filename (assuming the TFS Directory is full).

Note: By design, the extended-length file storage follows the highest file permitted in the TFS Directory. The present workaround actually allows 16 (0020 octal) additional blocks for the largest possible extended-length file as compared to the eventual formal redefinition. However, the extent of the TFS Directory can always be adjusted downward to allow more storage allocated to the extended-length file on any specific logical device unit. If desired, the TFS Directory on the device unit can be rendered moot causing all available storage past the last block reserved by the System Programs Directory to be available for the extended-length file. P?S/8 storage devices such as DECTapes can be configured in this manner for routine use with maximum size extended-length files. This is routinely used with programs such as P?S/8 *PAL* to process very large source files.

Appendix B (continued)

Details of the future implementation of extended-length file support include repurposing a few words at the end of the System Programs Directory for the extended-length file parameters. This will have minimal impact on the overall capacity of the System Programs Directory; in the worst case planned implementation, three less slots will be available for additional system programs (leaving a maximum of 82 program entries). As a practical matter, the System Programs Directory is presently utilized at under 40% of capacity; as such, this should be of no particular consequence for any intended usage.

Since the extended length file entry will be located in the System Programs Directory, it will have already been read in with the rest of the system programs support structure when the R (or RU{N}) form of execution is invoked. (The blocks overlaying a portion of the edit buffer include the few pages of code to process the entire command and the entirety of the System Programs Directory, which are located on adjacent blocks; as such, a single read operation sets up the entire command parsing process.)

Some of the underlying blocks currently assigned to the Extended-length Files Directory structure will be repurposed into part of a new low-level keyboard monitor used to manage all structures (such as the Logical Console Overlay and the P?S/8 SHELL) that wrest control from the keyboard monitor. This situation can become undesirable for a variety of reasons. For example, if the Logical Console Overlay is in control on a dismountable system, a serious problem could result should the system media be moved to a different machine lacking the key hardware. Additional blocks may be repurposed to other modules of the P?S/8 SHELL implementation such as the virtualized copy of the File and Communications area.

Once this updated form of program execution is fully implemented, all programs such as PTUTIL will be revised to fully support extended-length files with proper validation (meaning confirmation of starting file blocks required to exist beyond those within the TFS Directory structure that precedes it on the same logical device). In future releases, PTUTIL (and other conversion utilities) can check the TFS Directory structure on the same logical unit the extended-length file entry passed resides; the limit block of the TFS Directory defines the lowest block allowed for extended-length files. Thus, the passed file handle parameter should match this value.

Access to the dedicated entry in each system programs directory for the single extended-length file will require the user to set an appropriate unique filename not anomalous to any file within the associated TFS Directory on the same logical unit. If there is sufficient code space, the execution code will implement safeguards to ensure unique filenames by attempting to lookup the same filename within the associated TFS Directory.

Appendix B (continued)

Renaming of extended-length files will be implemented using new *SET* command options to establish proper naming conventions as well as preventing filename anomaly. Additional *SET* command options will manage scratch extended-length file area lists for programs such as P?S/8 PAL, which require large temporary files during the operation of the cross-reference module. (Up to eight extended-length files can be made available for reference storage during the assembly; active extended-length files resident on particular logical units must be excluded from this list to avoid destruction.)

End of Appendix B

Glossary of Terms

1. Edit buffer

TFS text files are created in the edit buffer using commands of the keyboard monitor. Files are tentative until the edit buffer contents are saved to a file in the TFS Directory.

2. Extended-length text files

Extended-length files partially resemble TFS text files except no actual line number or line pointer data exists. Additionally, there are no length restrictions on extended-length files (other than the physical limits of the device).

3. Extended-length Files Directory

This structure (which was never completed) is being removed in the next release of P?S/8. The original intention was to support multiple extended-length files; however this design was abandoned. Future releases will include the P?S/8 SHELL (currently under development). In P?S/8 Version 8Z, the only purpose of this structure is to establish the highest block that an extended-length file may write to.

In P?S/8 Version 8Z the system programs (such as *PTUTIL*, *OS8CON* and *L6DCON* that support writing extended-length files) are passed the starting block of extended-length files by a potentially dangerous kludge. For each such file, the starting block is specified by a TFS file that must be positioned high enough in the TFS Directory to prevent destruction of any TFS files potentially stored further up the device. The next release will eliminate this problem by specifying a single extended-length file explicitly using a special entry within the System Programs Directory; since the extended-length file can only be accessed via the R (or RU{N}) command (which reads in the entire System Programs Directory as well as the command processing code), overall system performance will actually improve when all related modifications are implemented.

4. Logical Console Overlay

The Logical Console Overlay is an optional component of P?S/8. When enabled, all conforming programs will direct all system console (and lineprinter) input/output to designated memory locations instead of the usual physical device. While the use of the Logical Console Overlay requires the use of extended memory, certain advanced features are enabled as follows:

Glossary of Terms (continued)

- 1) If the P?S/8 system device handler detects an error, the usual action is to halt the CPU (possibly displaying status bits in the accumulator); pressing `CONTINUE` on the system front panel will safely retry the latest read/write operation. Users need to be familiar with the particular system device handler at hand to gain insight as to what caused the specific problem (if possible) by examining the contents of the accumulator.

Note: An available system option is to disable the system halt which results in an infinite retry of all errors; depending on hardware particulars, this setting may not be recommended.

When the Logical Console Overlay is enabled, the action taken on detection of an error by the system device handler becomes a status report printed on the system console device (as defined by the Logical Console Overlay which could include alternate console hardware). This includes details of the latest read/write operation; the user can choose which action to take at the point of error by entering one of the stated options on the system console keyboard. This allows the user to correct the problem (if possible) and retry the latest read/write operation, or to abort the latest operation (if necessary). Without requiring specific knowledge of the vagaries of the current system device, any user can properly manage the system device operations.

- 2) Not every system device handler is capable of maximal performance; this is especially true of system device handlers designed to run on 4K systems (without requiring loading of a portion of the handler into extended memory). Due to limited memory space, it is often necessary to compromise performance to obtain basic functionality.

When the Logical Console Overlay is enabled, additional functionality (included in the system device handler for this purpose) is also enabled. This generally effects additional performance that overcomes the basic design compromise.

Note: On hardware configurations where the system device handler requires an extended-memory component, the Logical Console Overlay is designed to function in the rest of the same memory field. As such, for conforming programs attempting to locate extended memory fields, the use of the Logical Console Overlay does not change the number of complete memory fields available.

Glossary of Terms (continued)

- 3) Certain configurations of the Logical Console Overlay may use alternate console hardware (such as an alternate console terminal with device codes differing from 03/04). Future releases of PTUTIL will be able to detect the presence of these configurations; this will allow full message control during PTUTIL operations even if the low-speed reader/punch is selected.

5. System Programs Directory

In the basic P?S/8 system, the System Programs Directory structure is independent of all other directories. Every system program is stored in system blocks located prior to user files. Execution-class commands access the System Programs Directory to establish the memory loading/starting parameters for the specified program. Certain programs specify other attributes needed to perform special parsing rules that may apply. Every logical unit includes its own System Programs Directory; as such, in a multiple logical unit system configuration, numerous system programs may be available.

All system programs can specify additional attributes as follows:

- 1) Parsing of all files can be inhibited. This is required by programs such as *SET* which perform special parsing of the rest of the command line unrelated to filenames.
- 2) Parsing of input files can be inhibited. This is required by programs such as *BSAVE* which require output files while parsing the rest of the command line as memory ranges to save.
- 3) The *=xxxx* is normally processed as an octal value. If required, this can be forced to use the current radix in effect. This is useful for programs such as the *FIND* utility which interacts with the edit buffer using file line number values.

The System Programs Directory will be redesigned with the release of P?S/8 Version 9A. The new release will include explicit information on the extended-length file located past the TFS directory on the same logical unit.

Prior releases use the incomplete Extended-length Files Directory to determine the highest block allowed for writing to the extended-length file. This obsolescent structure was intended to support multiple files; however, it was never fully implemented. Future releases of P?S/8 will support the new P?S/8 SHELL structure (currently under development). The P?S/8 SHELL will support storage blocks past the extended-length file. (Note: The capacity of any of these structures can be adjusted downward to any storage size including zero; as such, it is possible to effectively have a device with only the SHELL structure active.)

Glossary of Terms (continued)

6. Tiny File System (TFS) Directory

This is the primary structure for user files in the basic P?S/8 system. Each TFS file is of fixed size allowing directory slots that cannot overlap other files in the overall structure. Projects can extend across multiple TFS files; the contents of all specified files will be logically concatenated together when processed by most system programs (such as P?S/8 *PAL*). Programs that create output files are aware of the implied file size (with the exception of certain programs such as *PTUTIL*, *OS8CON* and *L6DCON* which can optionally create extended-length files).

There are unique advantages to the TFS structure which, for certain users, requires some measure of orientation and experience. Once mastered, rapid editing of large projects can proceed with little additional effort.

Note: All P?S/8 system programs that create TFS files are aware of the specifics of the file structure; this includes binary files as created by P?S/8 *PAL* and P?S/8 *BIN* (when creating binary files from binary paper-tape input data).

7. Tiny File System (TFS) text files.

Text files created by the keyboard monitor are of fixed size. The lower addresses of the file store the actual text characters expanding towards the rear of the file. Line pointer and line number data are stored at the higher addresses within the file, expanding towards the front of the file. (The file is full when an attempt to add an additional text line would overflow the file while within the edit buffer; the actual file on a device is not affected until the edit buffer is saved to an actual TFS file.)

A system program that unpacks the text characters ignored the line pointer and line number values; the text processed is the concatenation of all files passed to the program in the order specified by the user within an execution-class command.

End of Glossary of Terms

[End-of-file]